Today

- Neural networks and deep learning
  - Single layer neural networks
  - Multi-layer neural networks
  - Convolutional neural networks
  - Other: transformers, U-net, RNN, LSTM

Innovations so far

- What innovations do we have so far that address the bias-variance tradeoff to give accurate out-of-sample predictions?
  - Average an ensemble
  - Randomly sample data (rows, eg bagging)
  - Randomly sample predictors (columns, eg RF)
  - Regularization (penalty terms, eg with splines)
  - Weak learners (eg shallow trees)
  - Slow learning (learning rate, eg boosting)

xgboost

- slow learning
  - gradient descent, boosting
- random sampling data
  - like bagging, but no replacement
- random sampling variables
  - like random forest but options (per tree, per node)
- regularized cost function
  - like smoothing spline, tree complexity penalty
- best split algorithm optimizes the whole tree
  - not per split

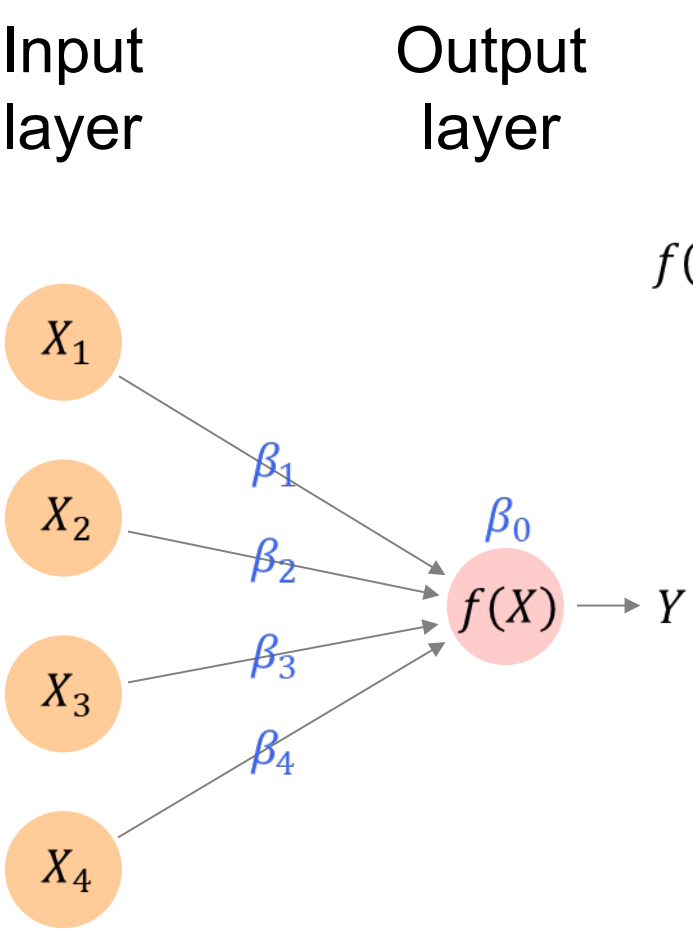
xgboost

- many additional algorithm innovations to speed training
- large data innovations
- inbuilt parallel and GPU support

Assignment tips

- Bagging improvement ca 1%
- Tune the tree for mincut first
  - Stabilize the k-fold noise: replicate partitions
  - Optimum is around 20 - 40
- Tune bagging algo for number of trees
  - Stabilize the k-fold noise: replicate partitions
  - Optimum is <50

Multiple linear regression

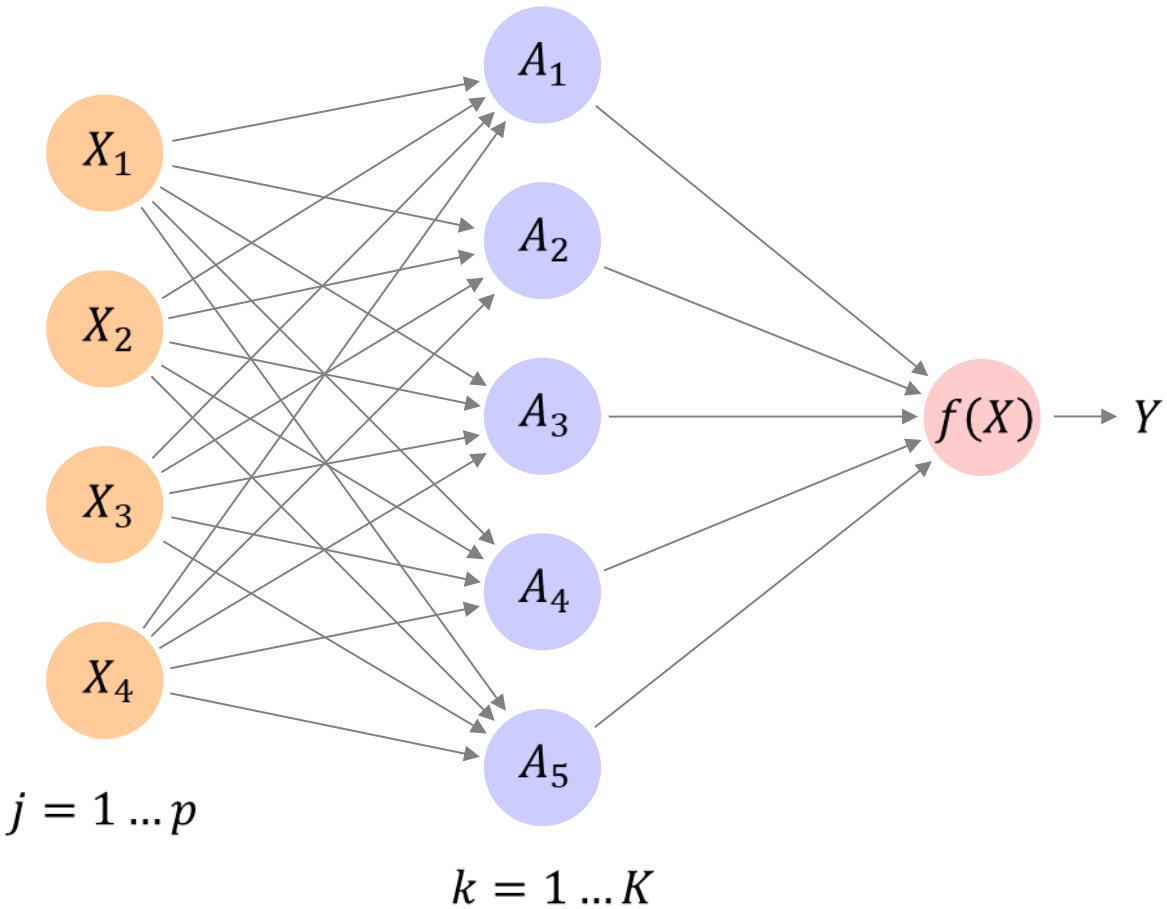


$$f(X) = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \beta_3 X_3 + \beta_4 X_4$$

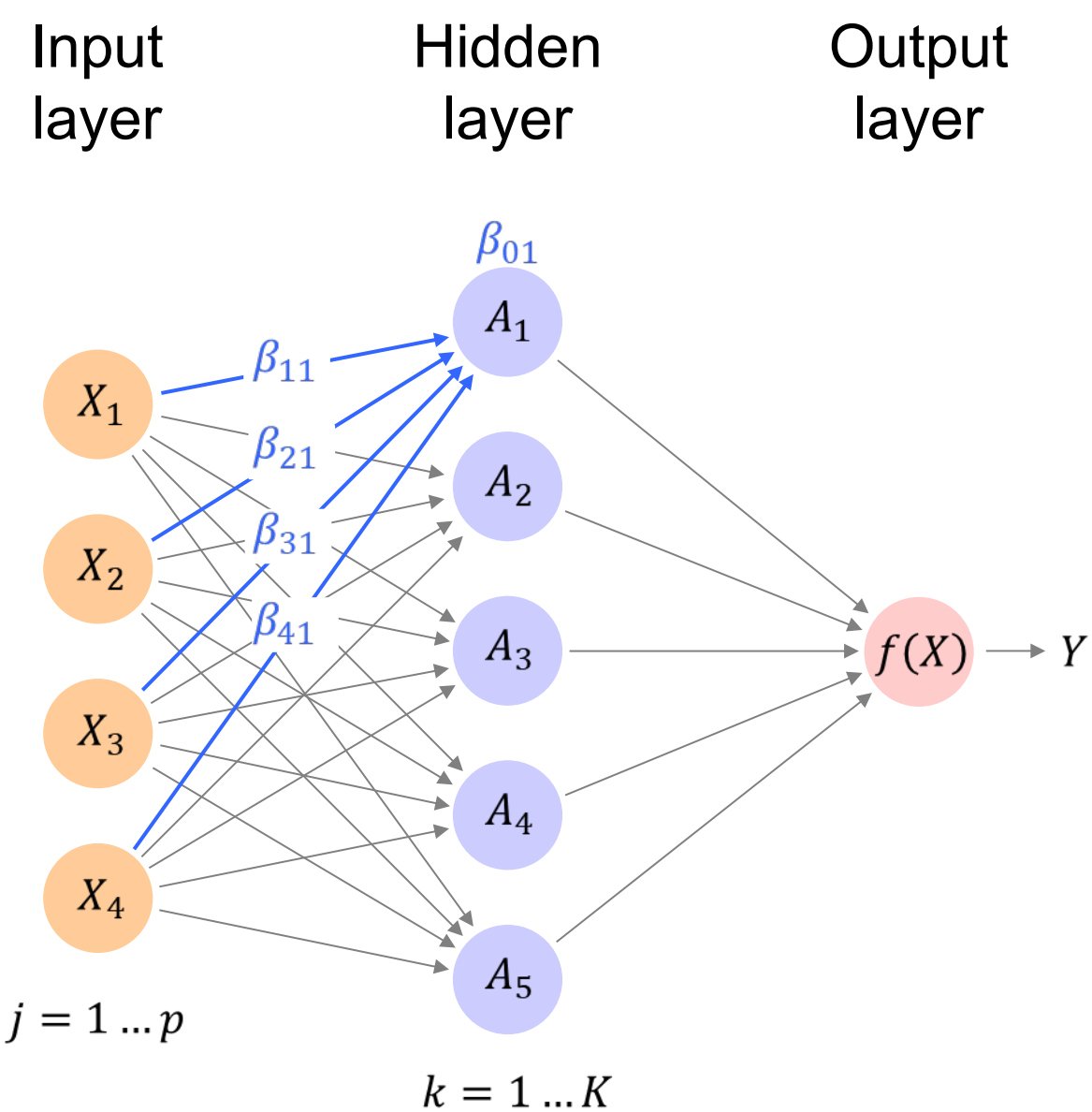
$$= \beta_0 + \sum_{j=1}^{p} \beta_j X_j$$

$j = 1 \ldots p$

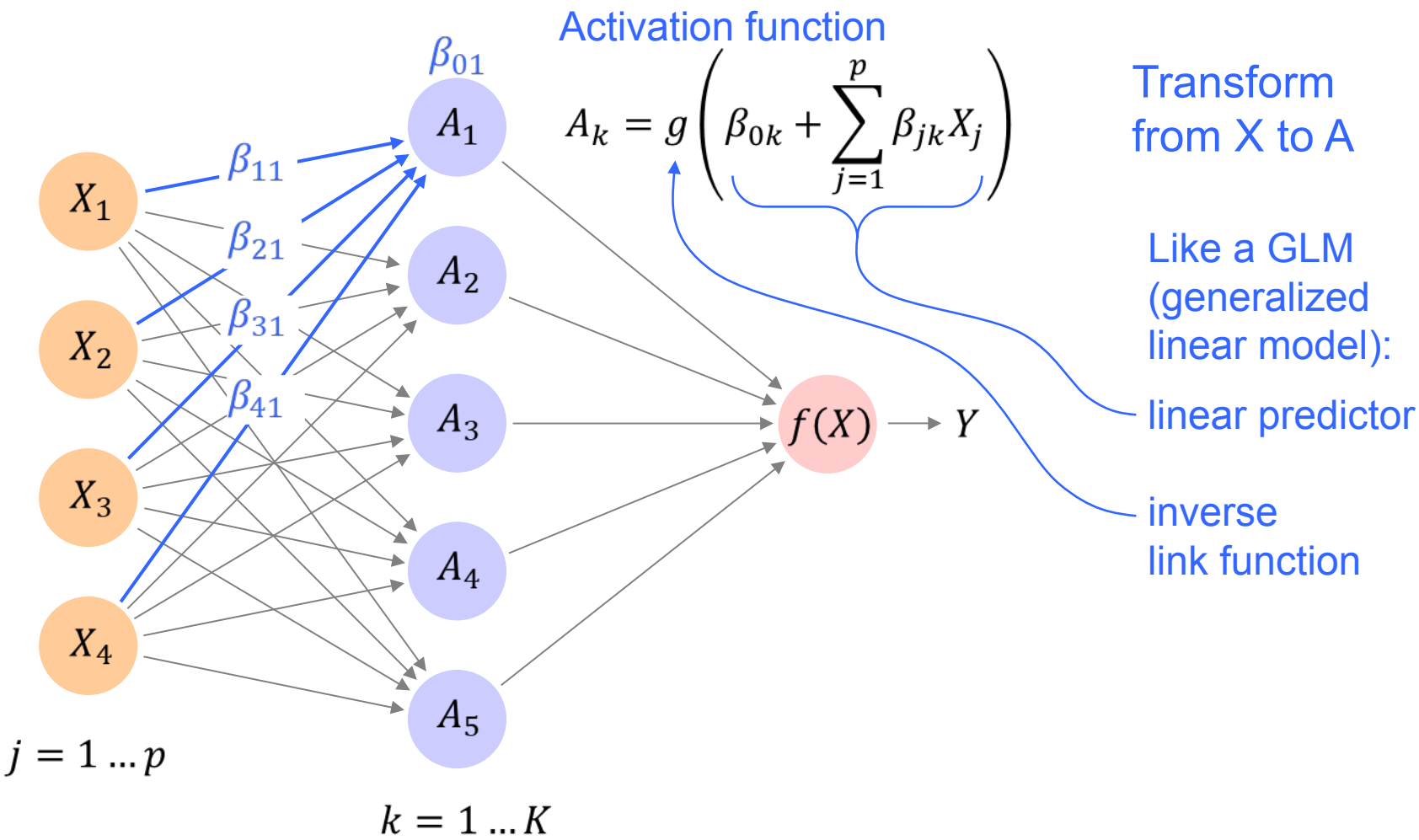
Single layer NN



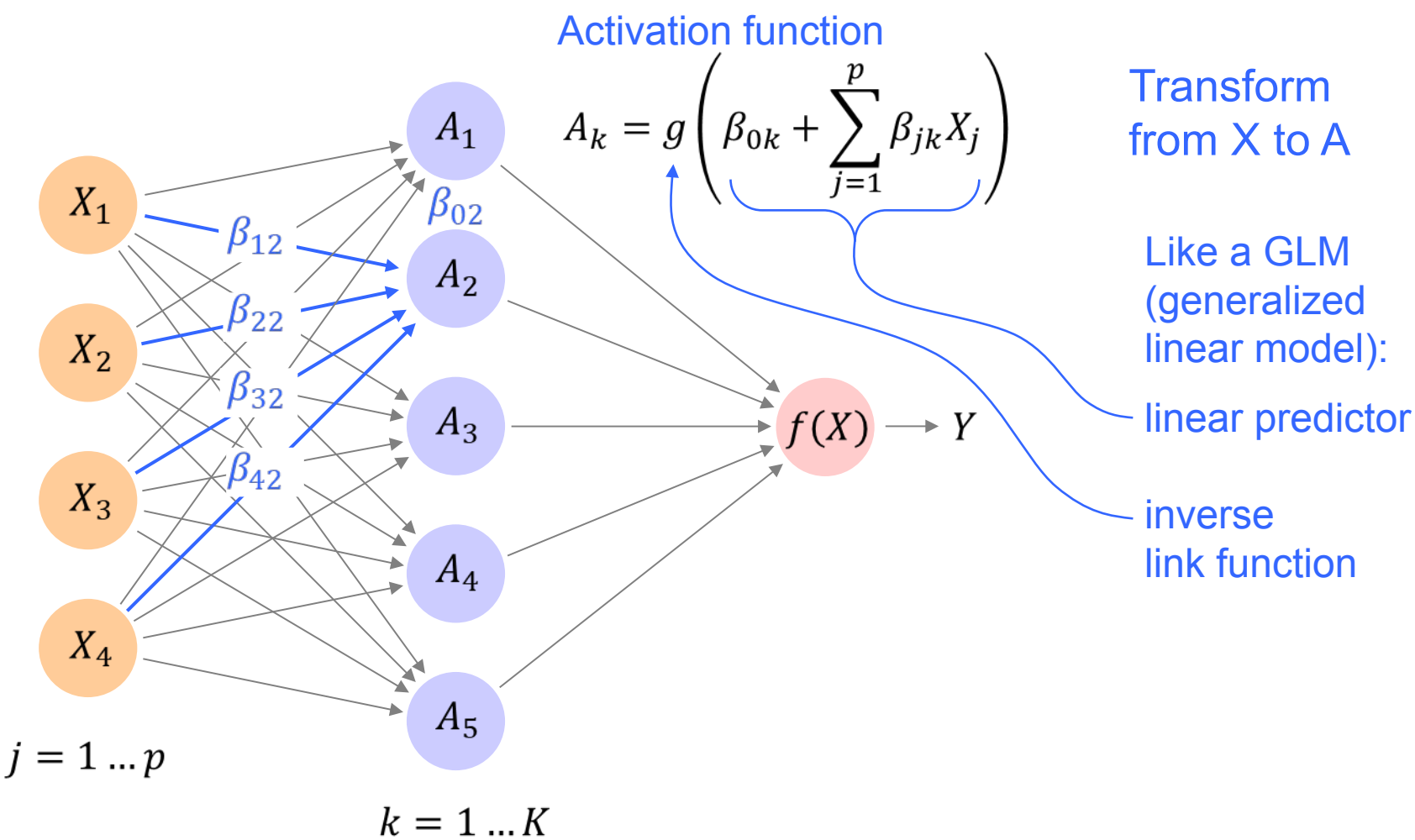
Single layer NN



Single layer NN



Single layer NN



Single layer NN

Input layer

Hidden layer

Output layer

Activation function

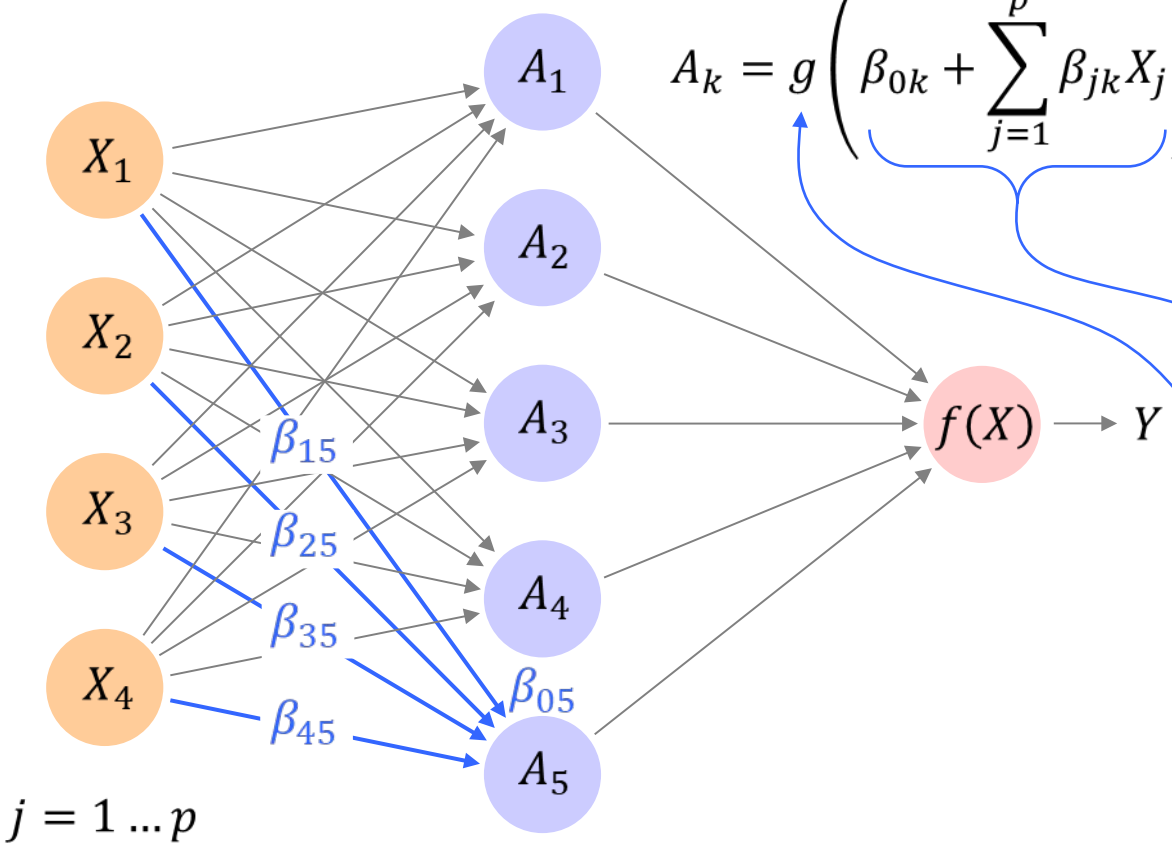
$$A_k = g\left(\underbrace{\beta_{0k} + \sum_{j=1}^{p} \beta_{jk} X_j}\right)$$

Transform from X to A

Like a GLM (generalized linear model):

linear predictor

inverse link function



Single layer NN

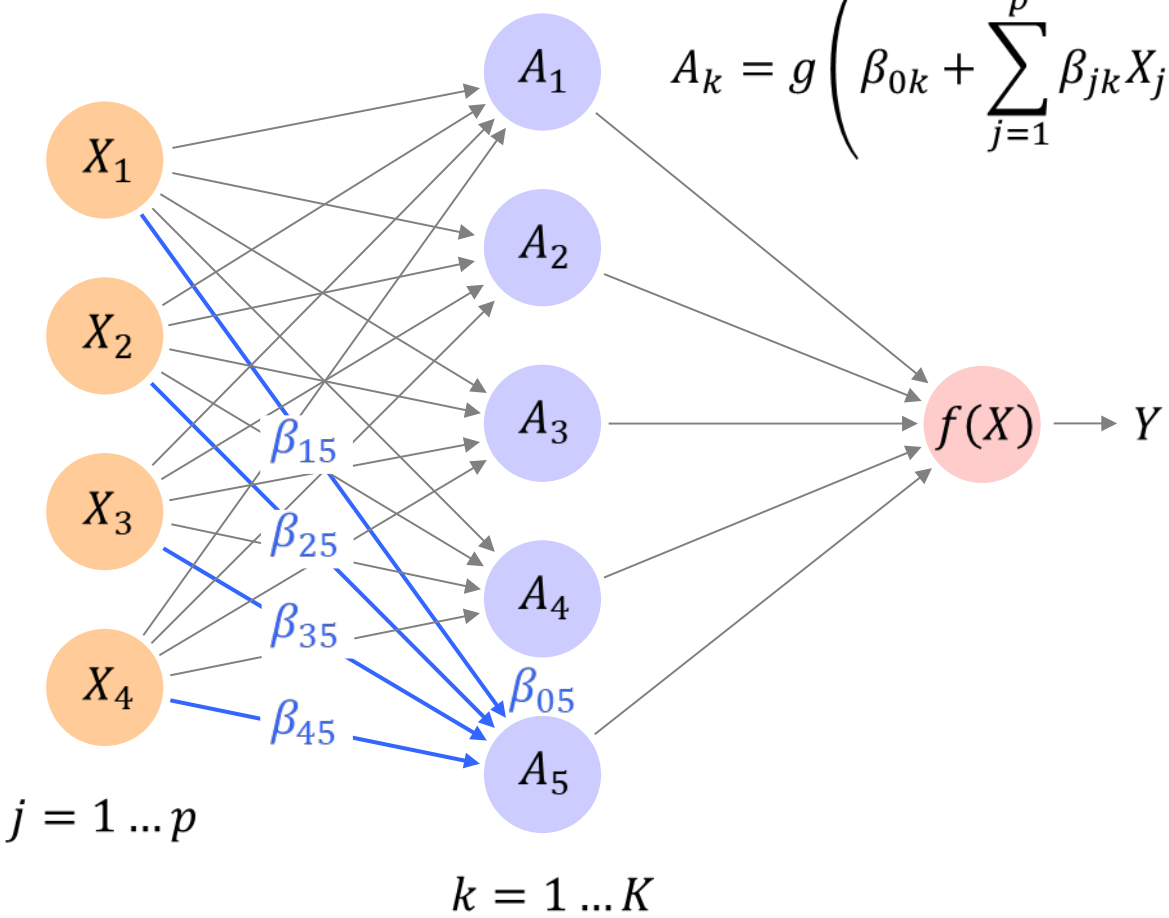
Input layer

Hidden layer

Output layer

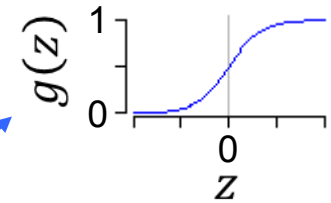
Activation function

$$A_k = g\left(\beta_{0k} + \sum_{j=1}^{p} \beta_{jk} X_j\right)$$

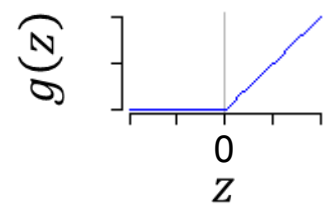


aka
(logit link)$^{-1}$

Sigmoid

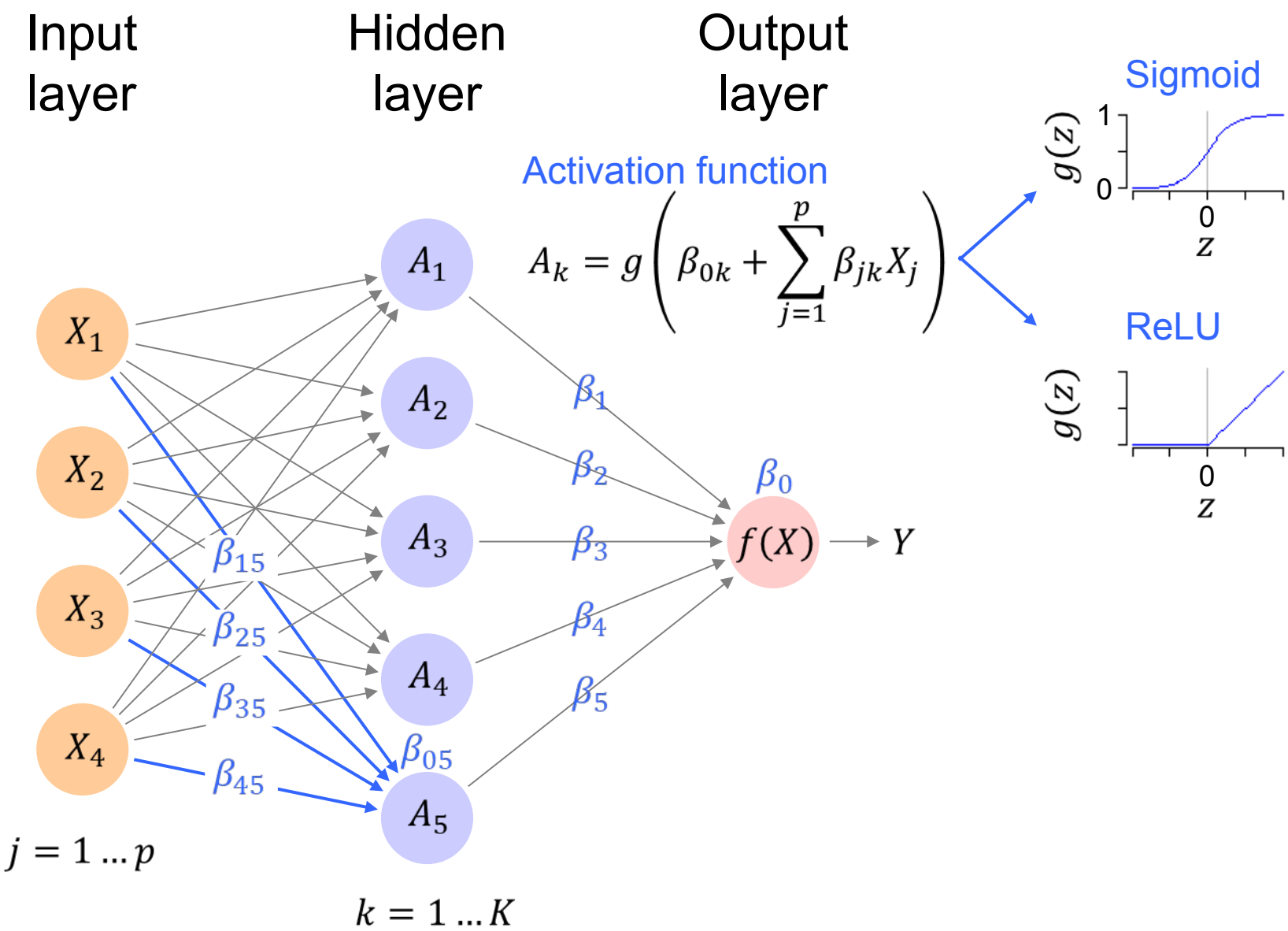


ReLU



Rectified linear unit

Single layer NN

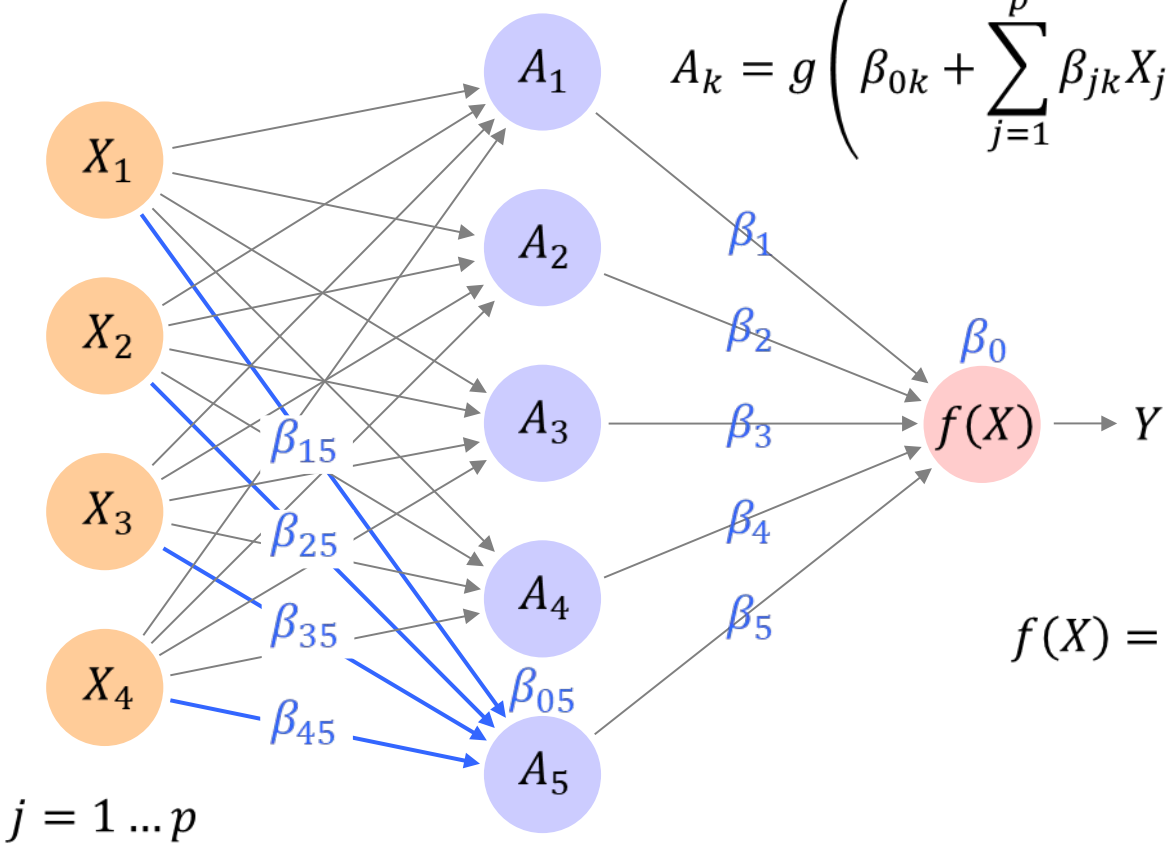


Single layer NN

Input layer

Hidden layer

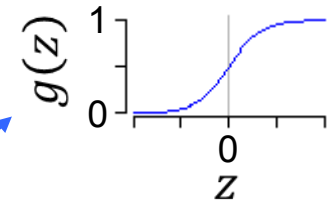
Output layer



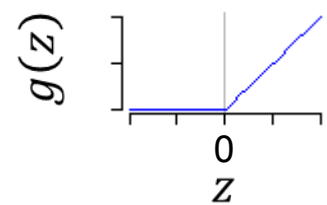
Activation function

$$A_k = g\left(\beta_{0k} + \sum_{j=1}^{p} \beta_{jk} X_j\right)$$

Sigmoid



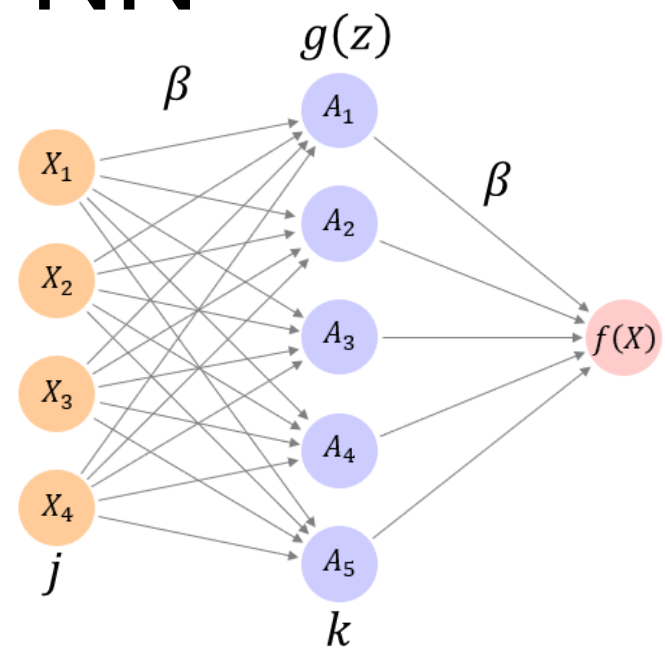
ReLU



$$f(X) = \beta_0 + \sum_{k=1}^{K} \beta_k A_k$$

multiple linear regression

Single layer NN



Model algorithm

define $g(z)$

load x_j

set K

set parameters: $\beta_{..}$

for each activation unit k in 1:K

calculate linear predictor: $z_k = \beta_{0k} + \sum_j \beta_{jk} x_j$

calculate nonlinear activation: $A_k = g(z_k)$

calculate linear model: $f(x) = \beta_0 + \sum_k \beta_k A_k$

return $f(x)$

Single layer NN (generalizing)

Input layer

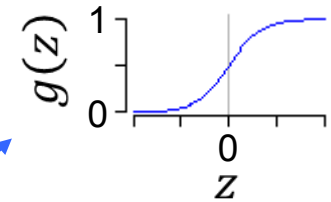
Hidden layer

Output layer

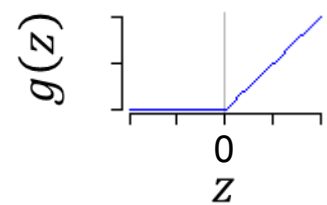
Activation function

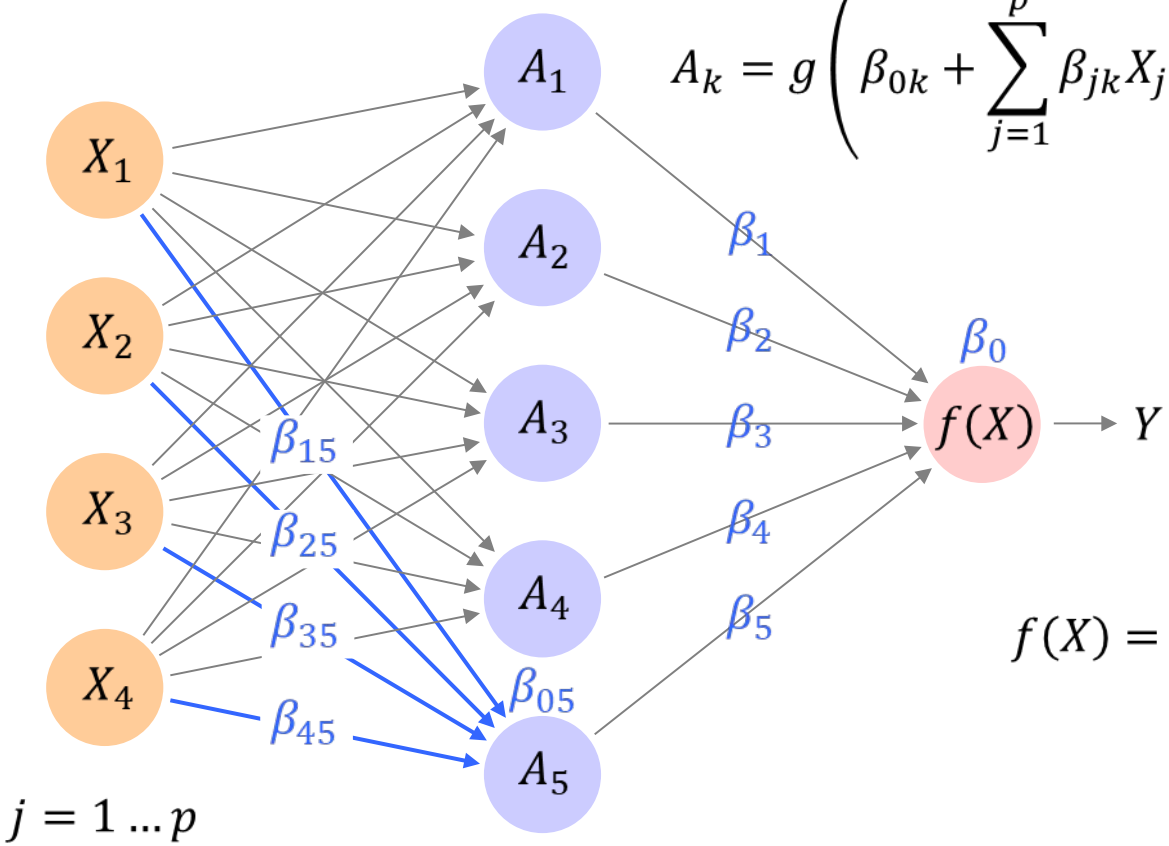
$$A_k = g\left(\beta_{0k} + \sum_{j=1}^{p} \beta_{jk} X_j\right)$$

Sigmoid



ReLU





$$f(X) = \beta_0 + \sum_{k=1}^{K} \beta_k A_k$$

Single layer NN (generalizing)

Input layer L_0

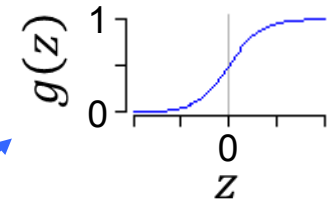
Hidden layer L_1

Output layer L_2

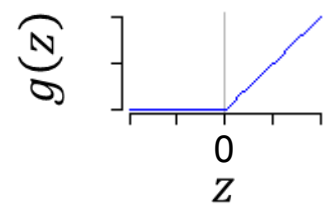
Activation function

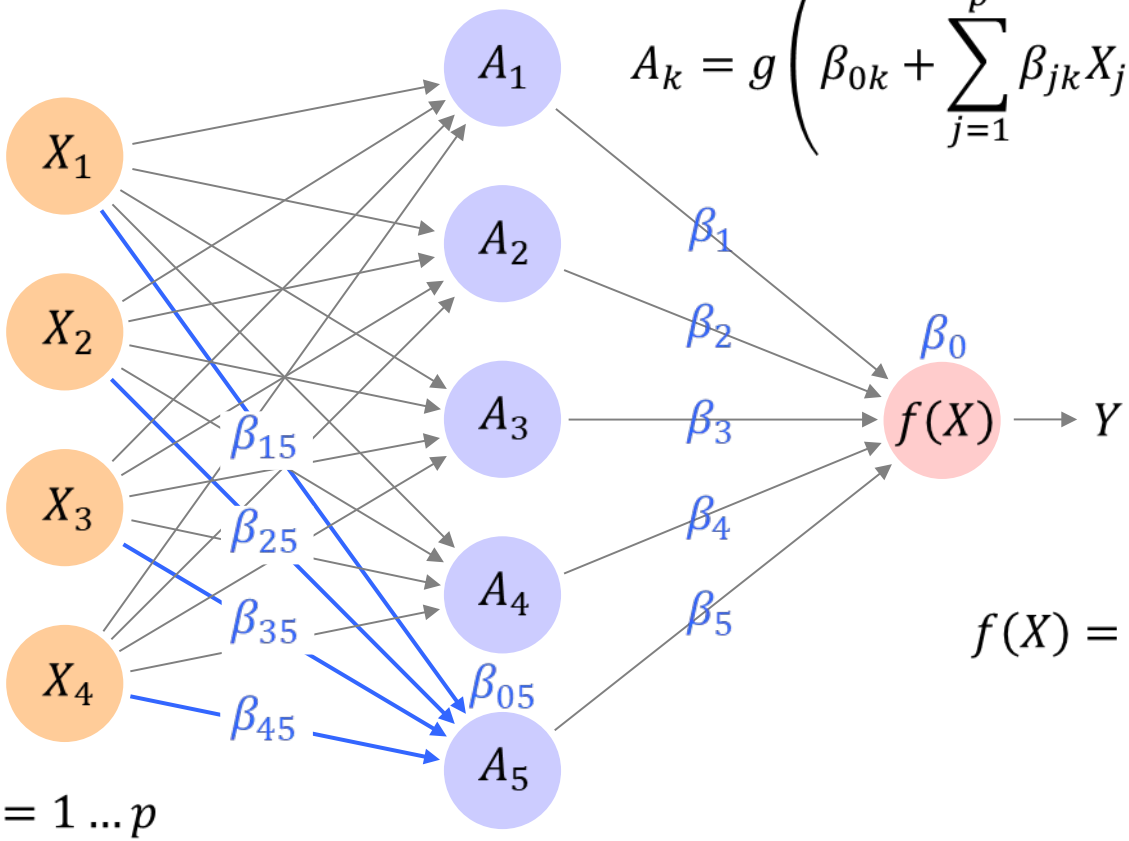
$$A_k = g\left(\beta_{0k} + \sum_{j=1}^{p} \beta_{jk} X_j\right)$$

Sigmoid



ReLU





$$f(X) = \beta_0 + \sum_{k=1}^{K} \beta_k A_k$$

$j = 1 \ldots p$

$k = 1 \ldots K$

Single layer NN (generalizing)

Input layer L_0

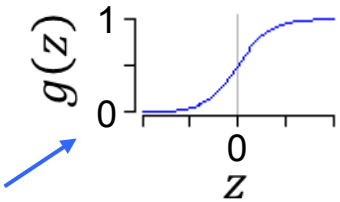
Hidden layer L_1

Output layer L_2

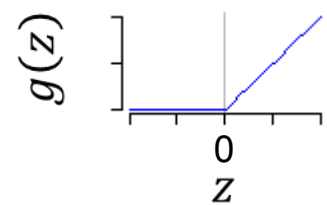
Activation function

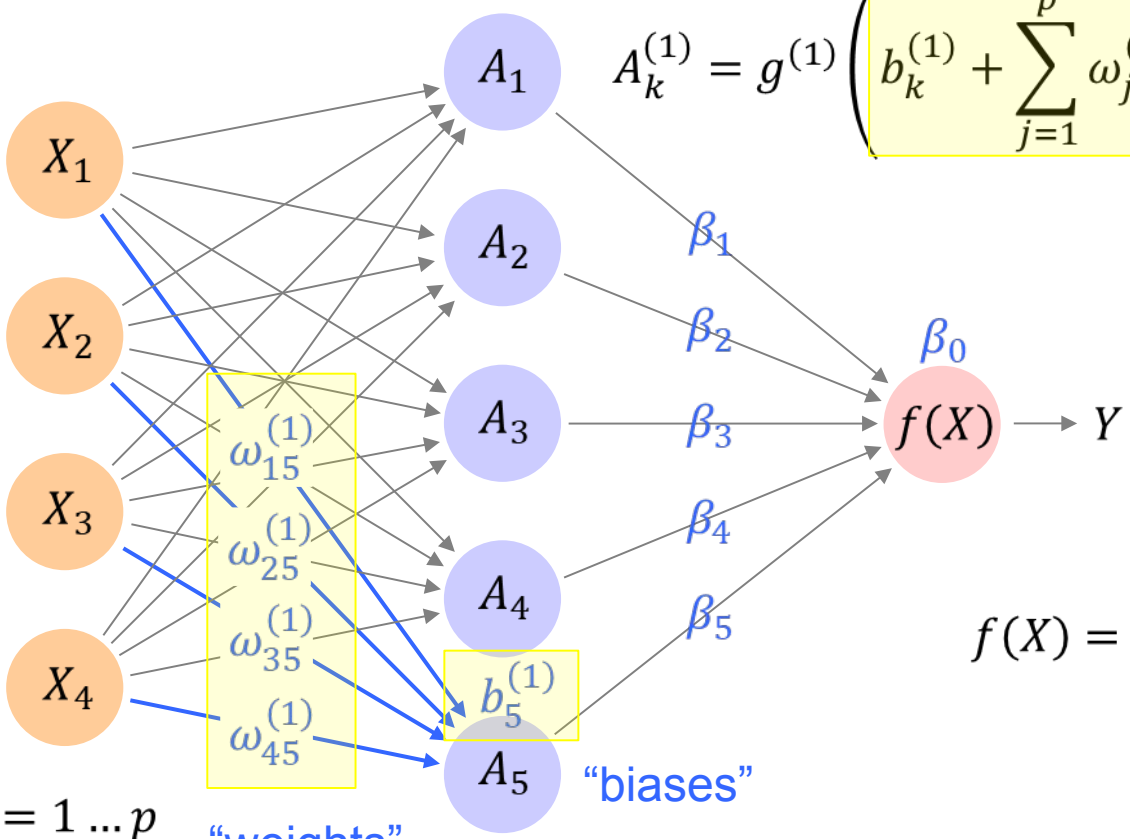
$$A_k^{(1)} = g^{(1)}\left(b_k^{(1)} + \sum_{j=1}^{p} \omega_{jk}^{(1)} X_j\right)$$

Sigmoid



ReLU





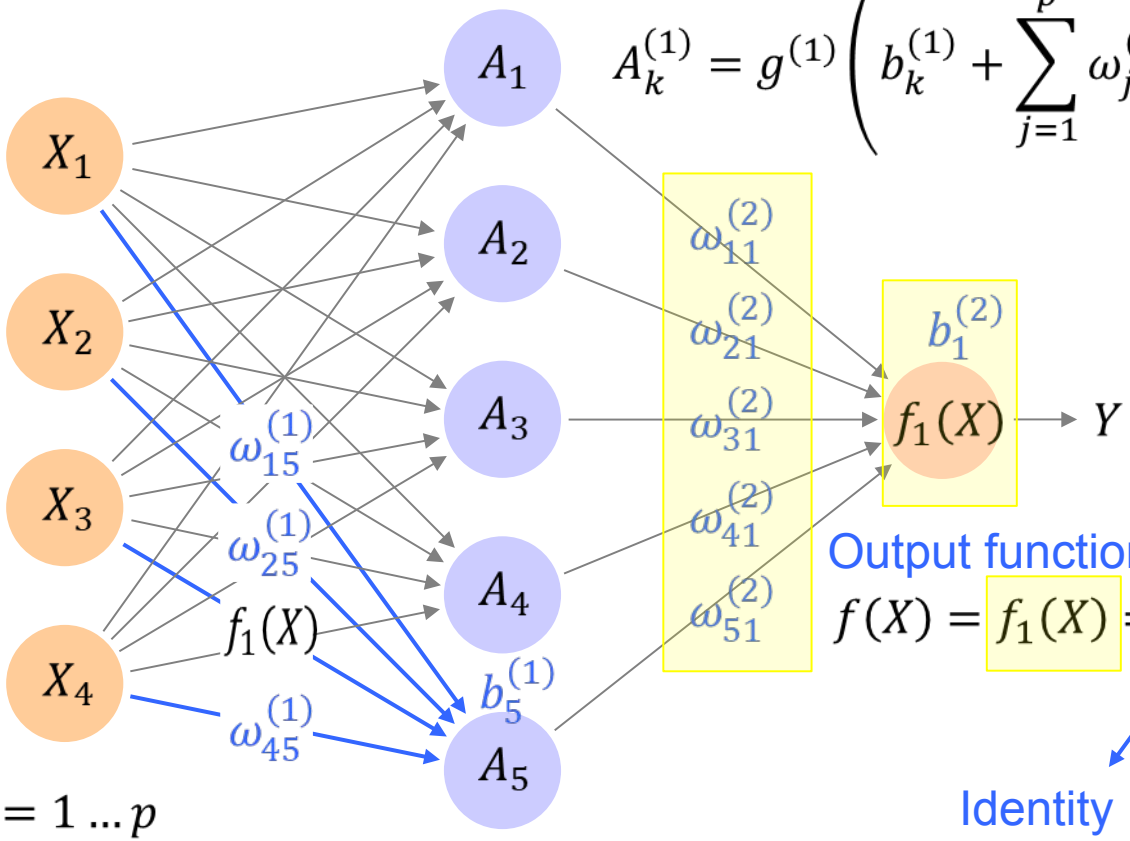
$$f(X) = \beta_0 + \sum_{k=1}^{K} \beta_k A_k$$

Single layer NN (generalizing)

Input layer L_0

Hidden layer L_1

Output layer L_2



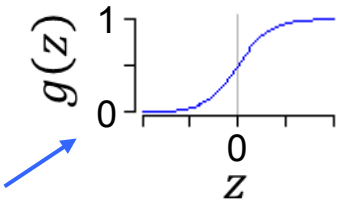
$j = 1 \dots p$

$k = 1 \dots K$

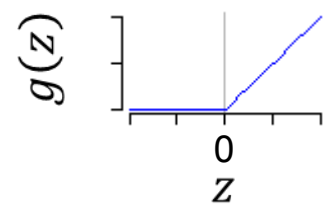
Activation function

$$A_k^{(1)} = g^{(1)}\left(b_k^{(1)} + \sum_{j=1}^{p} \omega_{jk}^{(1)} X_j\right)$$

Sigmoid



ReLU



Output function

$$f(X) = f_1(X) = g^{(2)}\left(b_1^{(2)} + \sum_{k=1}^{K} \omega_{k1}^{(2)} A_k\right)$$

Identity (Sigmoid) (Softmax)

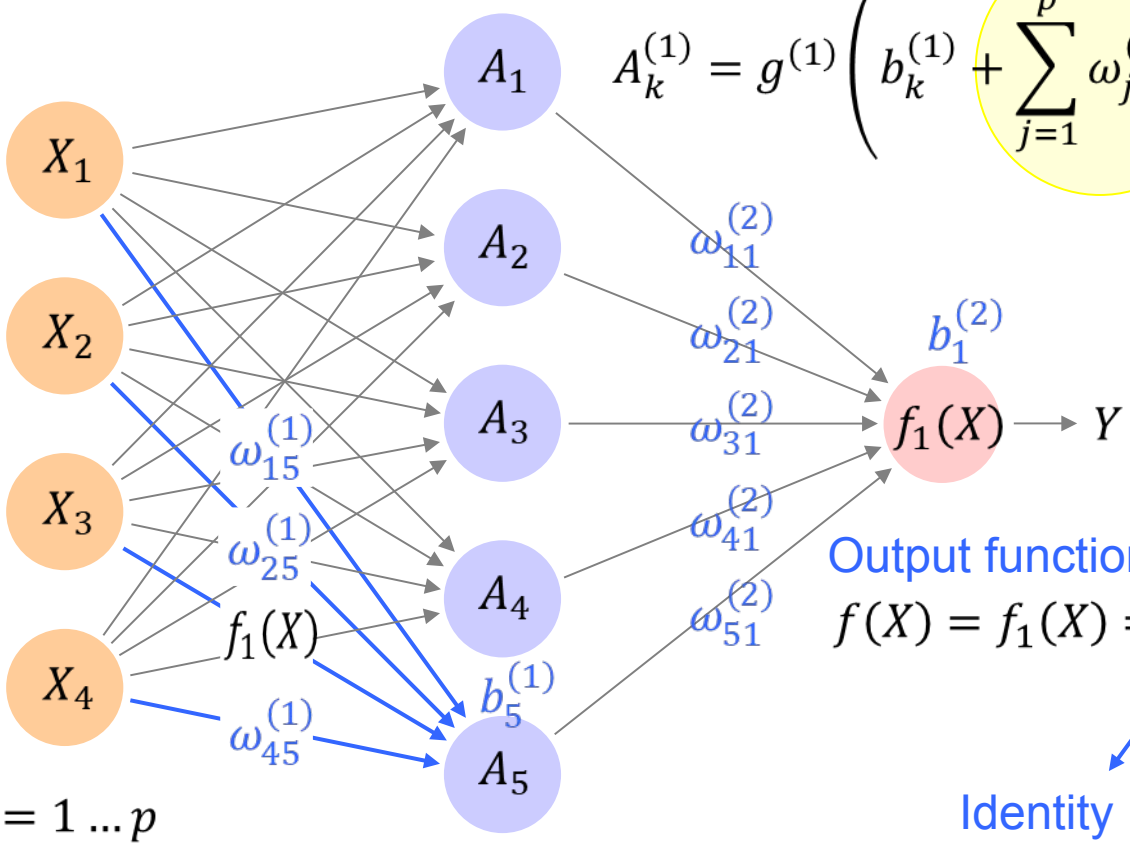
$g(z) = z$

Single layer NN (generalizing)

Input layer L_0

Hidden layer L_1

Output layer L_2



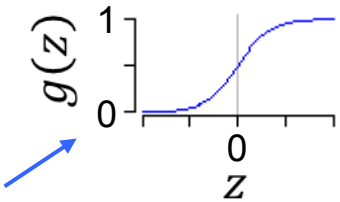
$j = 1 \dots p$

$k = 1 \dots K$

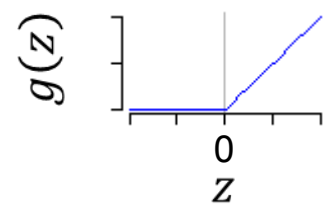
Activation function

$$A_k^{(1)} = g^{(1)}\left(b_k^{(1)} + \sum_{j=1}^{p} \omega_{jk}^{(1)} X_j\right)$$

Sigmoid



ReLU



Output function

$$f(X) = f_1(X) = g^{(2)}\left(b_1^{(2)} + \sum_{k=1}^{K} \omega_{k1}^{(2)} A_k\right)$$

Identity (Sigmoid) (Softmax)

$g(z) = z$

Single layer NN (generalizing)

$$\sum_{j=1}^{p} \omega_{jk}^{(1)} X_j$$

multiply down columns
then add across rows

$$\begin{array}{cccc} \omega_{1k} & \omega_{2k} & \omega_{3k} & \omega_{4k} \\ * & + * & + * & + * \\ X_1 & X_2 & X_3 & X_4 \end{array}$$

$$\begin{array}{cccc} x_{11} & x_{12} & x_{13} & x_{14} \\ x_{21} & x_{22} & x_{23} & x_{24} \\ \vdots & \vdots & \vdots & \vdots \\ x_{n1} & x_{n2} & x_{n3} & x_{n4} \end{array}$$

data rows
i = 1...n

data columns
j = 1...p

Matrix multiplication

$$\boldsymbol{X}\boldsymbol{\omega}_k$$

n x p p x 1

→ n x 1

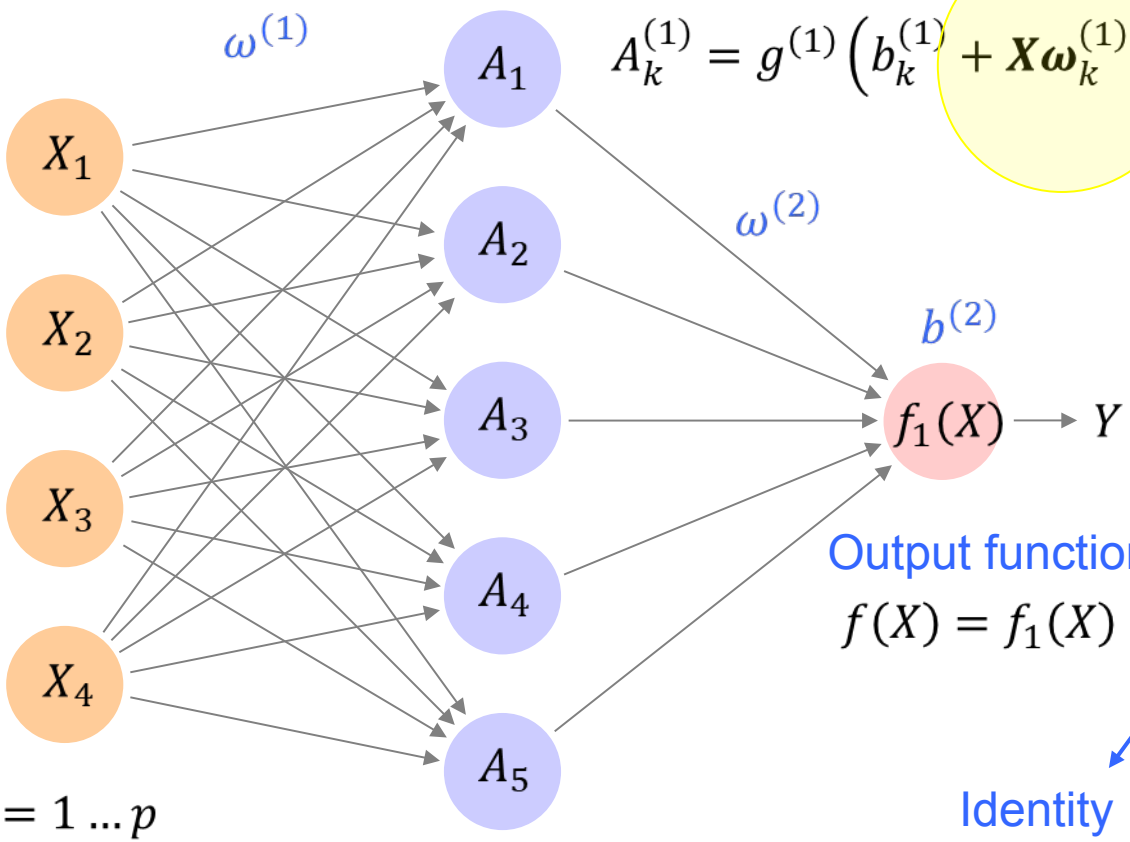
R: `x %*% w`

Single layer NN (generalizing)

Input layer L_0

Hidden layer L_1

Output layer L_2



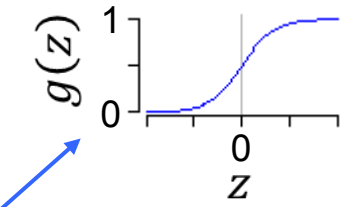
$j = 1 \dots p$

$k = 1 \dots K$

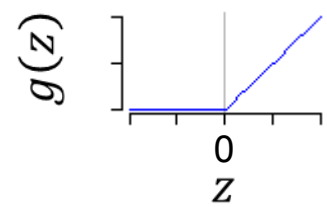
Activation function

$$A_k^{(1)} = g^{(1)}\left(b_k^{(1)} + \boldsymbol{X}\boldsymbol{\omega}_k^{(1)}\right)$$

Sigmoid



ReLU



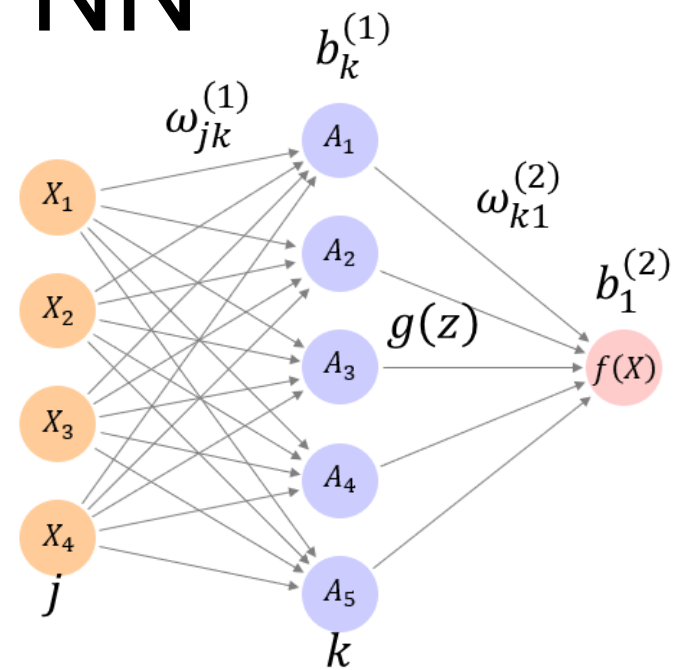
Output function

$$f(X) = f_1(X) = g^{(2)}\left(b_1^{(2)} + \boldsymbol{A}\boldsymbol{\omega}_1^{(2)}\right)$$

Identity (Sigmoid) (Softmax)

$g(z) = z$

Single layer NN



Model algorithm

define $g(z)$

load and prepare x_j

set K

set $\omega_{jk}^{(1)}$, $b_k^{(1)}$, $\omega_{k1}^{(2)}$, $b_1^{(2)}$

for each activation unit k in 1:K

calculate linear predictor: $z_k = b_k^{(1)} + \boldsymbol{X}\boldsymbol{\omega}_k^{(1)}$

calculate nonlinear activation: $A_k = g(z_k)$

calculate linear model: $f(x) = b_1^{(2)} + \boldsymbol{A}\boldsymbol{\omega}_1^{(2)}$

return $f(x)$

Code: ants_neural_net.R

Training algorithm

Loss (error) function (e.g. MSE)

$$\mathrm{MSE}(\theta) = \frac{1}{n}\sum_{i=1}^{n}\left(y_i - f_\theta(x_i)\right)^2$$

$$\theta = \begin{matrix} \beta_0 \\ \beta_1 \\ \vdots \\ \beta_{10} \\ \beta_{11} \\ \vdots \end{matrix}$$

Training algorithm

Stochastic gradient descent

guess θ (typically random)

set λ (learning rate)

for iterations (e.g. until $\mathrm{MSE}(\theta)$ stops decreasing)

randomly sample the data

calculate gradient of $\mathrm{MSE}(\theta)$: $\frac{\delta \mathrm{MSE}(\theta)}{\delta \theta}$ Method: back propagation

$$\theta \leftarrow \theta - \lambda \frac{\delta \mathrm{MSE}(\theta)}{\delta \theta}$$

Training algorithm

Stochastic gradient descent (mini batch)

guess θ (typically random)

set λ (learning rate)

for many epochs

    randomly partition data into batches

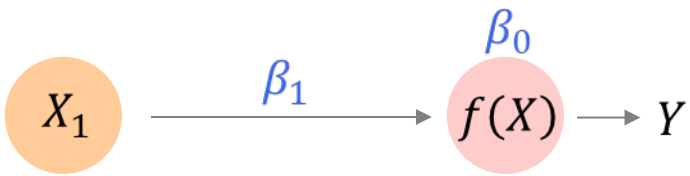
    for each batch

        calculate gradient of $\mathrm{MSE}(\theta)$: $\frac{\delta \mathrm{MSE}(\theta)}{\delta \theta}$

        $\theta \leftarrow \theta - \lambda \frac{\delta \mathrm{MSE}(\theta)}{\delta \theta}$

Training algorithm

e.g. simplest neural net!



$$f(X) = \beta_0 + \beta_1 X_1$$

Code: mini_batch_stoch_gradient_descent.R